

Veille technos Mobile

Veille Technologique Mobile — Flowly

Peut-on tout faire avec React Native ?

Non. La fonctionnalité de blocage d'applications est impossible avec React Native. Sur iOS, l'accès à **Screen Time API** (FamilyControls, ManagedSettings, DeviceActivity) est réservé aux apps Swift/SwiftUI natives avec un entitlement Apple spécifique. Sur Android, le monitoring d'apps via `UsageStatsManager` nécessite du code natif Kotlin/Java difficilement intégrable dans RN sans module custom lourd.

Technologies les plus adaptées

iOS → SwiftUI avec les frameworks natifs Apple :

- `FamilyControls` + `ManagedSettings` + `DeviceActivity` pour le blocage et le suivi d'activité
- `UserNotifications` pour les rappels
- `AVFoundation` + `MusicKit` pour le lecteur audio
- `AuthenticationServices` pour l'auth OAuth

Android → Flutter (Dart) :

- Accès aux APIs Android natives via `platform channels` (Kotlin)
 - `UsageStatsManager` pour le monitoring d'apps
 - `WorkManager` pour les tâches en arrière-plan
 - Meilleure alternative cross-platform que RN pour ce projet grâce au moteur de rendu propre (Skia)
-

Bibliothèque graphique

Oui, il en faut une. Flowly a une direction artistique forte (monde 3D, gamification, UI personnalisée) qui dépasse les composants natifs par défaut.

iOS (SwiftUI) :

- `Lottie` pour les animations (récompenses, streaks, effets de points)
- `Charts` (framework Apple natif iOS 16+) pour les statistiques
- `WKWebView` avec bridge JS pour afficher le monde Three.js

Android (Flutter) :

- `Rive` ou `Lottie Flutter` pour les animations
- `fl_chart` pour les statistiques
- `WebView Flutter` pour le monde Three.js

- Le moteur **Skia** de Flutter permet des animations et rendus custom très performants sans lib tierce

*Pour la DA spécifique, les deux plateformes nécessiteront des **composants custom** (cartes de shop, barre de progression gamifiée, avatar) — pas de lib toute faite qui correspondra exactement, il faudra les construire sur la base des outils ci-dessus.*

Architecture mobile la plus adaptée

MVVM (Model-View-ViewModel) sur les deux plateformes :

- SwiftUI est conçu autour de MVVM avec `@StateObject` / `@ObservableObject`
- Flutter s'y adapte bien avec `Provider`, `Riverpod` ou `Bloc`

C'est l'architecture la plus lisible, la plus testable, et la mieux documentée pour ces deux stacks.

Performances de React Native — suffisantes ?

Non, pas pour ce projet spécifiquement. Trois points bloquants : le pont JS/natif introduit de la latence sur les animations complexes, l'accès à Screen Time API est impossible, et le rendu custom de la DA sera plus difficile à optimiser. Flutter surpasse RN sur les performances graphiques grâce à son moteur de rendu indépendant.

Cross-platform ou natif distinct ?

Natif distinct (SwiftUI + Flutter) est le choix retenu et justifié. Le cross-platform (RN ou Flutter seul) ne couvre pas Screen Time API sur iOS. Flutter seul aurait pu être un compromis viable sur Android mais pas sur iOS pour cette fonctionnalité clé.

Accès aux fonctionnalités système

iOS :

- Notifications → `UNUserNotificationCenter`
- Arrière-plan → `BackgroundTasks` framework (`BGAppRefreshTask`)
- Permissions → déclarées dans `Info.plist`, demandées à runtime
- Stockage sécurisé → `Keychain` pour les tokens, `SwiftData` / `CoreData` pour les données locales

Android :

- Notifications → `NotificationManager` + `NotificationChannel`
 - Arrière-plan → `WorkManager` (tâches différées) + `Foreground Service` (session active)
 - Permissions → déclarées dans `AndroidManifest.xml`, demandées via `permission_handler` (Flutter)
 - Stockage sécurisé → `flutter_secure_storage` (EncryptedSharedPreferences sous le capot)
-

Cohérence UX entre iOS et Android

Le monde 3D, le leaderboard, la boutique et les stats étant affichés via **WebView commune**, la cohérence est naturelle pour ces écrans. Pour les écrans natifs, il faut définir un **design system partagé** (couleurs, typographie, espacements) et l'implémenter séparément dans chaque codebase — c'est le prix du natif distinct.

Différences de design entre les deux OS

Élément	iOS (SwiftUI)	Android (Flutter)
Navigation	<code>NavigationStack</code> , swipe back natif	<code>Navigator</code> / <code>GoRouter</code> , bouton retour Android
Modales	Sheet style iOS	Bottom sheet Material
Typographie	SF Pro (système)	Roboto (système)
Permissions	Demande unique avec explication	Demande runtime, peut être re- demandée

Sur Flutter, `Platform.isIOS` permet d'adapter les comportements si nécessaire. Sur SwiftUI c'est uniquement iOS donc pas de gestion de différences.

Structure du projet

SwiftUI :

```
Flowly/  
├── Core/           # modèles, services, networking  
├── Features/       # un dossier par feature (Focus, Shop, World, Leaderboard...)  
│   ├── Focus/  
│   │   ├── FocusView.swift  
│   │   ├── FocusViewModel.swift  
│   │   └── FocusModel.swift  
├── Shared/         # composants réutilisables, extensions  
└── Resources/      # assets, localization
```

Flutter :

```
lib/  
├── core/           # network, storage, theme  
├── features/       # feature-first  
│   ├── focus/  
│   │   ├── data/  
│   │   └── domain/
```

Gestion de l'état global

iOS (SwiftUI) → `@Observable` (iOS 17+) ou `@StateObject` + `@EnvironmentObject` pour partager l'état entre vues.

Simple, natif, pas de lib externe nécessaire.

Android (Flutter) → `Riverpod` : le plus moderne, le mieux typé, compatible avec l'architecture MVVM, et bien documenté. Alternative : `Bloc` si l'équipe préfère quelque chose de plus structuré.

Communication avec le back-end

Les deux apps communiquent via :

- **REST** pour les actions CRUD classiques (shop, profil, stats)
- **WebSocket / Supabase Realtime** pour la sync temps réel session desktop ↔ mobile

Sur iOS : `URLSession` (natif) ou `Alamofire`. Sur Flutter : `Dio` (lib HTTP la plus utilisée).

Gestion des erreurs, offline et perte de connexion

- **Mise en cache locale** : `SwiftData` (iOS) / `Hive` ou `Drift` (Flutter) pour persister les données essentielles
- **Retry automatique** sur les requêtes réseau avec backoff exponentiel
- **État offline visible** : bannière ou indicateur dans l'UI quand la connexion est perdue
- La session de focus peut continuer **hors ligne** (timer local) et se synchronise au retour de connexion

Sécurité des données

- Tokens stockés dans le **Keychain** (iOS) et **EncryptedSharedPreferences** (Android via `flutter_secure_storage`)
- **Certificate pinning** si les données sont sensibles
- Pas de données sensibles dans les logs
- Chiffrement des données locales via les APIs système (FileProtection sur iOS, EncryptedFile sur Android)
- **Expiration des tokens** gérée côté back-end + refresh token silencieux côté app

Déploiement et mises à jour

iOS : Xcode → TestFlight (beta) → App Store Connect. Nécessite un compte Apple Developer (99\$/an).

L'entitlement FamilyControls doit être demandé au préalable.

Android : Android Studio → Google Play Console. Compte à 25\$ une fois. Pipeline CI/CD recommandé :

Fastlane ou **GitHub Actions** pour les deux plateformes (build, sign, upload automatisés).

Maintenabilité sur le long terme

SwiftUI est le futur d'Apple, très bien maintenu et en amélioration constante. Flutter est soutenu par Google, cross-platform, avec une communauté très active. Les deux sont des choix pérennes. Le risque principal est la

duplication de logique entre les deux codebases — à mitiger en centralisant un maximum de logique dans le back-end.

Documentation et communauté

	SwiftUI	Flutter
Doc officielle	Excellente (developer.apple.com)	Excellente (flutter.dev)
Communauté	Grande (iOS dev worldwide)	Très grande et très active
Packages	Swift Package Index	pub.dev (très riche)
Ressources	WWDC videos, Hacking with Swift	Flutter.dev codelabs, Very Good Ventures

Conclusion — Meilleur compromis global

Le choix **SwiftUI (iOS) + Flutter (Android)** est le plus adapté à Flowly car il est le seul à permettre l'accès à Screen Time API sur iOS (fonctionnalité différenciante du projet), tout en offrant une expérience native de qualité sur Android avec Flutter. La DA custom et les animations de gamification sont mieux servies par ces deux stacks que par React Native. Le coût est une double codebase, compensé par une architecture MVVM commune et un back-end qui centralise la logique métier.